

Aula 06/15

Escola Tecnológica FPFtech

Curso: Automação para Indústria 4.0

Módulo: IoT Aplicado à Manufatura

Prof. Denis Moraes Guimarães

07 de abril de 2026

Sumário

- ▶ Leitura de umidade e temperatura
- ▶ PWM Led
- ▶ Comando Relé
- ▶ Leitura botão
- ▶ Comunicação peer to peer com Esp-Now

Leitura de umidade e temperatura

```
1 #include <DHT.h>
2
3 #define DHTPIN 4
4 #define DHTTYPE DHT11
5
6 DHT dht(DHTPIN, DHTTYPE);
7
8 void setup() {
9     Serial.begin(115200);
10    dht.begin();
11 }
12
13 void loop() {
14     delay(2000);
15     float umidade = dht.readHumidity();
16     float temperatura = dht.readTemperature();
17     if (isnan(umidade) || isnan(temperatura)) {
18         Serial.println("Erro ao ler o DHT11!");
19         return;
20     }
21     Serial.print("Umidade: ");
22     Serial.print(umidade);
23     Serial.print("  %\n");
24     Serial.print("Temperatura: ");
25     Serial.print(temperatura);
26     Serial.println("  C ");
27 }
```

PWM Led

```
1 #define LED_PIN 4
2
3 void setup(){}
4
5 void loop() {
6     for (int valor = 0; valor <= 255; valor++) {
7         analogWrite(LED_PIN, valor);
8         delay(10);
9     }
10
11     for (int valor = 255; valor >= 0; valor--) {
12         analogWrite(LED_PIN, valor);
13         delay(10);
14     }
15 }
```

Comando Relé

```
1 #define RELE_PIN 4
2
3 void setup() {
4     pinMode(RELE_PIN, OUTPUT);
5 }
6
7 void loop() {
8     digitalWrite(RELE_PIN, HIGH);
9     delay(2000);
10
11     digitalWrite(RELE_PIN, LOW);
12     delay(2000);
13 }
```

Obs: Efeito Bouncing

Leitura botão

```
1 #define BOTAO_PIN 4
2
3 void setup() {
4     Serial.begin(115200);
5     pinMode(BOTAO_PIN, INPUT_PULLUP);
6 }
7
8 void loop() {
9     int estado = digitalRead(BOTAO_PIN);
10
11     if (estado == LOW) {
12         Serial.println("Botao pressionado");
13     }
14     else {
15         Serial.println("Botao solto");
16     }
17
18     delay(200);
19 }
```

Comunicação peer to peer com Esp-Now

```
1 #include <esp_now.h>
2 #include <WiFi.h>
3
4 uint8_t peerMAC[] = {0x24, 0x6F, 0x28, 0xAA, 0xBB, 0xCC};
5
6 typedef struct struct_message {
7     int id;
8     int valor;
9 } struct_message;
10
11 struct_message mensagem;
12
13 esp_now_peer_info_t peerInfo;
14
15 // Callback de envio
16 void onDataSent(const uint8_t *mac_addr, esp_now_send_status_t status) {
17     Serial.print("Envio: ");
18     Serial.println(status == ESP_NOW_SEND_SUCCESS ? "OK" : "Erro");
19 }
```

Comunicação peer to peer com Esp-Now

```
1 // Callback de recebimento
2 void onDataRecv(const uint8_t * mac, const uint8_t *incomingData, int len) {
3     struct_message recebido;
4
5     memcpy(&recebido, incomingData, sizeof(recebido));
6
7     Serial.print("Recebido de: ");
8
9     for (int i = 0; i < 6; i++) {
10         Serial.printf("%02X", mac[i]);
11         if (i < 5) Serial.print(":");
12     }
13
14     Serial.print(" | valor: ");
15     Serial.println(recebido.valor);
16 }
```


Comunicação peer to peer com Esp-Now

```
1 void setup() {
2     Serial.begin(115200);
3
4     WiFi.mode(WIFI_STA);
5
6     Serial.print("Meu MAC: ");
7     Serial.println(WiFi.macAddress());
8
9     if (esp_now_init() != ESP_OK) {
10        Serial.println("Erro ESP-NOW");
11        return;
12    }
13
14    esp_now_register_send_cb(onDataSent);
15    esp_now_register_recv_cb(onDataRecv);
16
17    memcpy(peerInfo.peer_addr, peerMAC, 6);
18    peerInfo.channel = 0;
19    peerInfo.encrypt = false;
20
21    if (esp_now_add_peer(&peerInfo) != ESP_OK) {
22        Serial.println("Erro ao adicionar peer");
23        return;
24    }
25 }
```

Comunicação peer to peer com Esp-Now

```
1  void loop() {  
2      mensagem.id = 1;  
3      mensagem.valor = random(0, 100);  
4  
5      esp_now_send(peerMAC, (uint8_t *) &mensagem, sizeof(mensagem));  
6  
7      Serial.print("Enviado: ");  
8      Serial.println(mensagem.valor);  
9  
10     delay(2000);  
11 }
```

Quiz de Revisão

